

Aztec Labs Polynomial Audit Summary

Prepared by Cyfrin

Version 2.0

Lead Auditors

Farouk

qpzm

May 8, 2026

Contents

1 About Cyfrin **2**

2 About This Document **2**

3 Disclaimer **2**

4 Copyright and License **2**

5 Audit Overview **3**

6 Audit Scope **3**

7 Risk Classification **4**

8 Protocol Summary **4**

8.1 Module File Overview 4

8.2 Storage Model 4

8.3 Core Polynomial Operations 5

8.4 Types Used by Sumcheck 5

8.5 Evaluation Domain and FFT 5

9 Attestation **5**

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 About This Document

This document is a public summary of the security audit Cyfrin performed for Aztec Labs on Polynomial. It records the repositories and commits reviewed, the timeline, auditors, in-scope files, and the number and final status of findings by severity. It closes with Cyfrin's attestation of the review's scope and final remediation status.

It is deliberately abbreviated: the technical detail of individual findings is not reproduced here. The complete audit report, version 2.0 dated May 8, 2026, was delivered to Aztec Labs, who retain it as the full record of the engagement. This summary was issued on October 29, 2026.

3 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited. Unless the scope explicitly notes otherwise, the review is limited to the application layer and does not extend to the correctness of API/SDK/execution environment implementations, compilers, standard and third-party libraries, underlying operating systems and hardware.

4 Copyright and License

© 2026 Cyfrin Inc. All rights reserved. Material from Aztec Labs or third parties remains the property of its owners.

This summary may be shared only in complete and unmodified form, so that any copy can be checked against the hash in its attestation. Short quotations with attribution to Cyfrin are permitted, provided they do not misrepresent this summary's scope or results. Except as permitted by applicable law, no part of this summary may otherwise be copied, modified, adapted, sold or sublicensed, or used to create derivative works, without Cyfrin's prior written permission.

Cyfrin and the Cyfrin logo are trademarks of Cyfrin Inc. and may not be used to suggest that Cyfrin has reviewed or endorses anything outside this summary's scope, or to misrepresent its results.

5 Audit Overview

Over the course of 8 days, the Cyfrin team conducted an audit on the Polynomial code provided by Aztec Labs. The findings consist of 15 Low severity issues with the remainder being informational.

Summary

Project Name	Polynomial
Repository	aztec-packages
Commit	e88ecdf581db...
Audit Timeline	Apr 13th - Apr 22nd, 2026
Methods	Manual Review

Issues Found

Severity	Found	Resolved	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	15	15	0
Informational	8	4	4
Gas Optimizations	0	0	0
Total	23	19	4

6 Audit Scope

The audit scope was limited to:

```
aztec-packages/barretenberg/cpp/src/barretenberg/polynomials/backing_memory.cpp
aztec-packages/barretenberg/cpp/src/barretenberg/polynomials/backing_memory.hpp
aztec-packages/barretenberg/cpp/src/barretenberg/polynomials/barycentric.hpp
aztec-packages/barretenberg/cpp/src/barretenberg/polynomials/eq_polynomial.hpp
aztec-packages/barretenberg/cpp/src/barretenberg/polynomials/evaluation_domain.cpp
aztec-packages/barretenberg/cpp/src/barretenberg/polynomials/evaluation_domain.hpp
aztec-packages/barretenberg/cpp/src/barretenberg/polynomials/polynomial.cpp
aztec-packages/barretenberg/cpp/src/barretenberg/polynomials/polynomial.hpp
aztec-packages/barretenberg/cpp/src/barretenberg/polynomials/polynomial_arithmetic.cpp
aztec-packages/barretenberg/cpp/src/barretenberg/polynomials/polynomial_arithmetic.hpp
aztec-packages/barretenberg/cpp/src/barretenberg/polynomials/shared_shifted_virtual_zeroes_array.hpp
aztec-packages/barretenberg/cpp/src/barretenberg/polynomials/univariate_coefficient_basis.hpp
```

7 Risk Classification

	Impact: High	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

8 Protocol Summary

The `polynomials` module is the foundational data-structure and arithmetic layer of Barretenberg's UltraHonk proving system. It provides the polynomial types, field-arithmetic primitives, evaluation domains, and lookup-table precomputations that Sumcheck, Gemini, Shplonk, KZG, IPA, and the ZK-Sumcheck Libra protocol all build on.

Every UltraHonk proof uses this module. Wire/selector/permutation polynomials are instances of `Polynomial<Fr>`; Sumcheck's round univariates are `UnivariateCoefficientBasis` + `Univariate`; the multilinear equality polynomial in Sumcheck is `ProverEqPolynomial` / `VerifierEqPolynomial`; Gemini's fold polynomials, Shplonk's batched quotient, and IPA's final reduction are built from the `Polynomial` arithmetic operations. ECCVM and Translator (Grumpkin curve) use the same types plus the `SmallSubgroupIPA` + `BarycentricData`-backed Libra protocol for ZK. The module is explicitly instantiated only for `bb::fr` (BN254 scalar) and `grumpkin::fr`.

8.1 Module File Overview

File	Role
`backing_memory.cpp/hpp`	Heap and optional file-backed mmap allocator for polynomial storage
`shared_shifted_virtual_zeroes_array.hpp`	Sparse array primitive: memory-backed over `[start_, end_)`, logically valid over `[0, virtual_size_)` with zeros outside the backed range
`polynomial.cpp/hpp`	Main `Polynomial<Fr>` type: coefficient arithmetic, MLE evaluation, shifts, reversals, interpolation
`polynomial_arithmetic.cpp/hpp`	Free-function primitives: Horner evaluation, FFT/iFFT (BN254 only), Lagrange interpolation, `factor_roots`, linear product
`evaluation_domain.cpp/hpp`	FFT domain with precomputed roots-of-unity tables
`barycentric.hpp`	Compile-time and runtime barycentric interpolation data
`univariate_coefficient_basis.hpp`	Degree ≤ 2 polynomials in monomial basis with a Karatsuba-friendly encoding for Sumcheck relations
`eq_polynomial.hpp`	Multilinear equality polynomial `eq(X, r)` (prover full-table form and verifier single-point form)

8.2 Storage Model

1. `BackingMemory<Fr>` owns the raw field-element buffer, allocated either on the heap or as a file-backed `mmap`.
2. `SharedShiftedVirtualZeroesArray<T>` holds `start_`, `end_` and `virtual_size_` plus a `BackingMemory<T>`. Only indices in `[start_, end_)` consume memory; reads outside that range return zero. This sparse layout lets Honk represent shifted wire polynomials and polynomials with many trailing zeros without allocating the unused regions.
3. `Polynomial<Fr>` wraps one `SharedShiftedVirtualZeroesArray<Fr>` and provides the public API: constructors, MLE and Horner evaluation, in-place `+=`/`-=`/`*=`, `add_scaled`, `shifted()`, `reverse()`, `full()`, `factor_roots()`, interpolation, and the `share()` shallow-copy primitive.

8.3 Core Polynomial Operations

Operation	Function(s)	Protocol use
MLE evaluation	`Polynomial::evaluate_mle(u, shift)`	Sumcheck opening step (prover and verifier via Gemini)
Univariate Horner evaluation	`Polynomial::evaluate(z)`	Gemini fold-polynomial evaluations; KZG opening-pair evaluations
Synthetic division by `(X - r)`	`Polynomial::factor_roots(r)`	KZG opening quotient; Shplonk per-claim quotient; Goblin merge prover
Left-shift (divide by X)	`Polynomial::shifted()`	Honk shifted-witness polynomials; Gemini fold polynomials
Multi-polynomial batching	`operator+=`, `operator-=`, `add_scaled`	Gemini batch-polynomial construction; Shplonk batched quotient; Sumcheck round polynomial accumulation
Lagrange interpolation	`compute_efficient_interpolation`, `compute_linear_polynomial_product`	SmallSubgroupIPA challenge polynomial; ZK-Sumcheck Libra concatenated polynomial
FFT / iFFT	`fft_inner_parallel`, `ifft`	ZK-Sumcheck Libra monomial-form conversion (BN254)

8.4 Types Used by Sumcheck

- `UnivariateCoefficientBasis` holds degree ≤ 2 polynomials $a_0 + a_1 X + a_2 X^2$ in monomial basis and can cache $a_0 + a_1$ to support a Karatsuba-style multiplication.
- `Univariate` is the Lagrange-basis counterpart over the consecutive-integer domain $\{0, 1, \ldots\}$; extension to larger domains uses direct formulas for small lengths and `BarycentricData` otherwise.
- `BarycentricData` precomputes the domain, Lagrange denominators and related products used for barycentric extension and evaluation.
- `ProverEqPolynomial::construct(r, log_num_monomials)` builds the full 2^d-entry coefficient table of

$$\mathrm{eq}(X, r) = \prod_{i=0}^{d-1} \big((1 - r_i)(1 - X_i) + r_i X_i\big).$$

- `VerifierEqPolynomial::evaluate(u)` computes $\mathrm{eq}(u, r)$ at a single point in $O(d)$ without constructing the full table.

8.5 Evaluation Domain and FFT

`EvaluationDomain<Fr>` packages everything needed to run an in-place radix-2 Cooley-Tukey FFT over a multiplicative subgroup of size $n = 2^k$: the primitive n-th root of unity and its inverse, a coset generator and its inverse, and the precomputed root tables. Real FFTs are used with BN254 `fr`, where the ZK-Sumcheck Libra subgroup has size 256. Grumpkin `fr` has 2-adicity 1, so the ECCVM/Translator Libra path uses direct Lagrange interpolation over an 87-element multiplicative subgroup instead of an iFFT.

9 Attestation

Cyfrin attests that it performed a security review of the Polynomial code provided by Aztec Labs.

The repositories, commits, review period and findings with their final status are in the Summary and Issues Found tables in Section 5, and the in-scope files in Section 6. This attestation is limited to the documented scope, audited commits and mitigations specifically reviewed by Cyfrin.

Cyfrin publishes this document and its SHA-256 hash at https://github.com/Cyfrin/cyfrin-audit-reports. To check a copy on macOS or Linux, run the following in the folder that contains it; it prints `OK` when the copy matches. If the copy was renamed, set F to its file name. Before running it, check that the URL is Cyfrin's repository.

```
U=https://github.com/Cyfrin/cyfrin-audit-reports/raw/main/reports/sha256
N=2026-05-08-cyfrin-aztec-polynomial-v2.0.pdf
F=$N
H=$(curl -fsSL $U/$N.sha256 | cut -d' ' -f1)
echo "$H *$F" | shasum -a 256 -c
```

Cyfrin's full audit report with all findings provided to Aztec Labs has the SHA-256 hash and file name below. To check a copy of it, first check this document as above, then run the following in the folder that contains the full report; it prints `OK` when the copy matches. If the copy was renamed, set F to its file name.

```
H=9b5600e094ff50f7a67862b853d4dbeb47d5abc282f59f883ba4e8e121eebe73
F=2026-05-08-cyfrin-aztec-polynomial-v2.0-full.pdf
echo "$H *$F" | shasum -a 256 -c
```

On Linux, `sha256sum -c` can be used in place of `shasum -a 256 -c`. On Windows, run `Get-FileHash <file> -Algorithm SHA256` in PowerShell and compare the result with the hash.